SETTING THE COOKIE FROM CORS

The rules described in this section also apply to setting the cookie from the server. If the `withCredentials` property and Access-Control-Allow-Credentials header are both `true`, the server can set a cookie on the client. This cookie still can't be read from JavaScript code, but it will be included on subsequent requests to the server.

This section introduced a lot of new code to both the server and the client. Here is a recap:

1. The login.html page sets a cookie identifying the user.
2. The client sets the `withCredentials` property to indicate it would like to send cookies on CORS requests.
3. The server validates the user in the cookie before deleting a post.
4. The server sets the Access-Control-Allow-Credentials header to indicate that cookies are okay on CORS requests.

By including both a client- and server-side component, the developer has a lot of flexibility and control over when cookies are included in requests.

5.2 Exposing response headers to the client

All the CORS-specific headers introduced so far handle how an incoming request behaves. The Access-Control-Allow-Methods header specifies the valid request methods, while the Access-Control-Allow-Headers header specifies the valid request headers. Now we'll turn our attention to the HTTP response behavior. Specifically, you'll learn how client JavaScript code can access the response headers.

By now you may be discerning a pattern to how CORS works: for the client to do anything, the server must first give its permission. This behavior extends to the headers in the HTTP response. Response headers are the headers the server sends back to the client. Figure 5.7 shows a request from the sample's client.html page to the API. The response headers are the headers from the API back to the client.

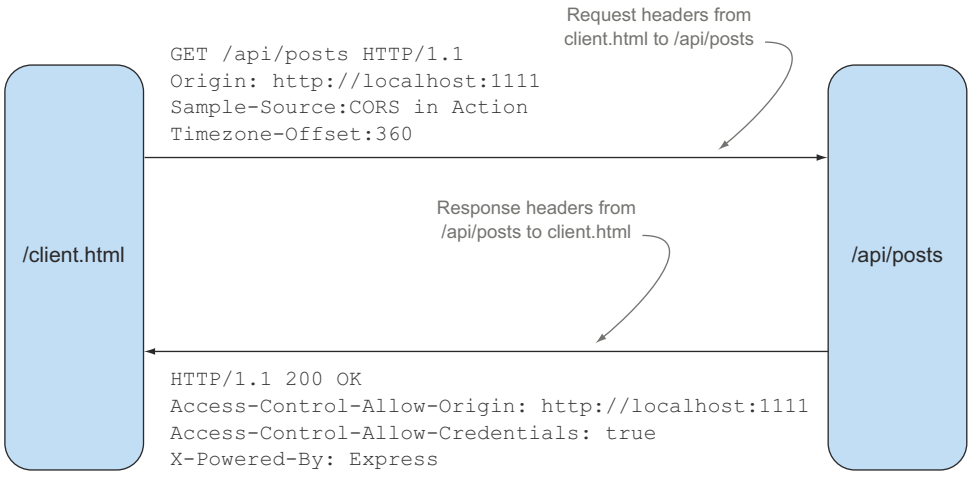


Figure 5.7 The distinction between request and response headers